UNIVERSITY OF GHANA

DEPARTMENT OF COMPUTER SCIENCE

SEMESTER II 2025/2026 ACADEMIC YEAR

ASSIGNMENT 1

DSCD 614 – Reinforcement Learning

Programming Assignment 1: Frozen Lake from First Principles Using Q-Learning

Name: Dau-Mensah Kwaku
Student ID: 22425187

TECHNICAL REPORT

Introduction

Reinforcement Learning is a unique branch of machine learning that focuses on teaching an agent how to make optimal decisions through trial and error. Instead of being fed a static dataset with pre-labelled answers, the agent actively interacts with a dynamic environment to discover which actions yield the best long-term outcomes. This learning process mirrors how humans and animals naturally adapt to their surroundings. For example, it is very much like teaching a dog tricks using treats; when the dog performs the correct action, it receives a reward, which reinforces that positive behaviour and makes it more likely to repeat it in the future.

From a technical standpoint, this framework studies how an agent can use its past experiences and historical data to enhance the future control of a changing, complex system. Every time the agent makes a move, the environment transitions into a new state and sends back a numerical feedback signal, which can be a positive reward for success or a penalty for a mistake. The agent's ultimate goal is to map these situations to the best possible actions so that it can maximize its total accumulated rewards over time. By balancing the need to explore new, unknown pathways with the desire to exploit its best-known safe routes, the agent gradually builds a robust master plan to solve the task at hand. A practical application of the concept of Reinforcement Learning is the Frozen Lake puzzle game.

This assignment seeks to apply these core concepts and principles of Reinforcement Learning to solve the classic **Frozen Lake** grid-world puzzle. The agent acts as a walker trying to find a safe path across an 8x8 frozen grid, starting from a designated safe space (S) and aiming for a cabin goal tile (G). Along the way, the agent must learn to completely avoid stepping on lethal water holes (H), which terminate the game instantly with zero reward, while safely stepping across standard frozen ice tiles (F). Since the agent receives no guidance other than a final reward upon reaching the goal, it must rely entirely on its learning algorithm to master the terrain.

Environment Design

In designing the environment, a custom FrozenLakeEnv class was built entirely from scratch using standard Python without importing any external reinforcement learning libraries. The environment defines a static 8×8 grid world consisting of 64 distinct tiles. The 2D grid coordinates are mapped into a 1D state representation so as to allow for easy compatibility of the grid with a standard tabular structure. Each square is assigned a single unique integer index ranging from 0 in the top-left corner to 63 in the bottom-right corner. This integer state is calculated dynamically using a row-major conversion formula: **State = (Row x 8) + Column**. The environment class tracks the agent's current position, prevents it from moving off the edges of the board by enforcing strict grid boundaries, and automatically detects when the agent enters a terminal tile.

The agent interacts with this world by executing one of four discrete directional choices at each step, where action 0 moves the agent Left, 1 moves Down, 2 moves Right, and 3 moves Up. The reward system is designed to guide the agent toward the safe completion of the puzzle while heavily punishing mistakes. The agent receives a positive reward of +1.0 only when it successfully navigates to the Goal tile (G). Moving onto normal frozen ice tiles (F) provides a

neutral reward of 0.0, ensuring that the agent does not receive intermediate credit just for walking around. If the agent falls into a dangerous water hole (H), it receives a reward of 0.0 and the episode terminates instantly. Because the environment features sparse rewards, the agent must successfully reach the final goal layout before it receives any positive mathematical feedback to reinforce its path choices.

Q-Learning Agent/Algorithm

Q-Learning is a model-free, value-based temporal difference algorithm designed to discover the optimal action-selection policy for any given Markov Decision Process. In the assignment, the algorithm is implemented from first principles using a flat tabular data format known as a Q-table. This table functions as a memory grid where rows represent the 64 discrete states of the frozen lake and columns represent the 4 possible directional actions. The numbers stored inside the table, called Q-values, represent the expected cumulative future reward the agent will receive if it performs a specific action in a specific square. At the start of the training phase, the entire table is initialized to zero, meaning the agent begins with absolutely no knowledge of the map or the consequences of its moves.

The core learning process occurs step-by-step as the agent interacts with the grid and updates its table values using the exact Bellman optimality equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)$$

In the mathematical formula, the learning rate parameter is set to 0.1, which dictates that new sensory experiences will gently blend with past estimations without erasing previous knowledge. The discount factor is configured at 0.95 to balance the weight of short-term and long-term feedback; this high value ensures the agent prioritizes finding a complete pathway to the ultimate goal over thousands of steps rather than just seeking safety in the immediate next square. To balance discovering new pathways with using established routes, we implemented an Epsilon-Greedy exploration strategy. A tracking probability variable, called epsilon, dictates whether the agent explores a random, unknown path or exploits the highest available numerical value in its current Q-table to ensure steady convergence.

Training Methodology

The training process compared two exploration setups over 15,000 games. The Decaying Epsilon Agent started with a full exploration rate of 1.0, which dropped by a multiplier of 0.995 after each game until hitting a 0.01 floor. This drop allowed the agent to explore early on and then switch to using its best-known paths. The Pure Epsilon Agent used a fixed exploration rate locked at 0.20 for all games. Running these two agents side-by-side showed exactly how changing the exploration strategy impacts learning speed and path stability.

During the training loop, the performance was closely monitored by logging four key statistics at every step. The total rewards for each episode was recorded to see when the agent reached the goal. A rolling window was utilized to count successful runs and calculate a real-time success rate percentage. Finally, the epsilon values for each game were saved to watch the exploration changes over time. This close data tracking proved that the decaying strategy helps the agent find the best solution much faster than the static approach.

Experimental Results and Analysis

After completing the full training phase, both agents were subjected to a rigorous evaluation phase consisting of 100 consecutive trial runs where all exploratory behaviour was disabled (epsilon = 0.0). In this evaluation phase, both the Decaying Epsilon Agent and the Pure Epsilon Agent achieved a perfect **100.00% Success Rate**, an **Average Reward of 1.00**, and exactly **zero(0) failures**. This perfect score demonstrates that both exploration strategies allowed their respective Q-learning models to thoroughly discover and calculate the mathematically optimal path from the start tile to the goal. Once the random noise of exploration was removed, both models strictly exploited the highest values in their completed Q-tables, navigating around every lethal water hole on the 8x8 grid without a single error.

However, looking at the logged training telemetry reveals a stark difference in how these two agents behaved during the learning process. The Decaying Epsilon Agent learned the layout remarkably fast, reaching a highly stable 99.5% success rate by episode 2,000. Because its epsilon value continually decayed, it rapidly phased out random actions, allowing it to lock onto and repeatedly reinforce the shortest safe path. Conversely, the Pure Epsilon Agent hovered around a lower 78% to 83% success rate throughout the entire training cycle. Because its exploration rate was permanently locked at 20%, it was mathematically forced to make blind, random moves even after it knew the correct path, causing it to repeatedly tumble into water holes. These contrasting learning curves were automatically plotted as an ASCII performance bar graph and saved inside “results/performance_chart.txt”, providing clear empirical proof that a decaying cooling schedule yields superior training stability.

Learned Policy Extraction and Analysis

Following the completion of the training cycle, the final target policy was extracted directly from the maximum state-action values of the completed Q-table. This process matches the standard decision rule where the recommended action for any given state is chosen by finding the index with the highest numerical value. The complete grid policy is mapped out textually below, using directional arrows to represent the clear paths discovered by the best-performing agent across the 8x8 grid layout:

==========

| → | → | → | ↓ | ↓ | ← | ? | ? | (Row 0)

| ↑ | ← | ↑ | → | → | ↓ | ↓ | ↓ | (Row 1)

| ? | ? | ? | H | → | ↓ | ← | ← | (Row 2)

| ? | ? | ? | H | ↑ | → | ↓ | ↑ | (Row 3)

| ? | ? | ? | H | ? | → | → | ↓ | (Row 4)

| ? | H | H | ? | ? | ? | H | ↓ | (Row 5)

| ? | H | ? | ? | H | ? | H | ↓ | (Row 6)

| ? | ? | ? | H | ? | ? | ? | G | (Row 7)

==========

Keys: → - Left → - Right ↑ - Up ↓ - Down H - Hole G - Goal ? - Unvisited

An analysis of this grid map layout reveals a highly efficient navigation path that cleanly works around the layout constraints. The agent starts in the top-left tile at state 0 and immediately drives eastward along Row 0, before safely looping down and around the vertical line of dangerous water holes (H) located in Column 3. The presence of question mark symbols (?) on the final grid layout indicates unvisited states that still hold their initial zero value matrix entries. Because the agent discovered a safe path to the goal early in the training phase, its exploration rate dropped before it ever needed to step on those distant tiles. This shows excellent algorithmic efficiency; the agent optimized its route along a safe pathway and avoided wasting computation on irrelevant or dangerous parts of the map.

Challenges Encountered

- **Sparse Rewards:** The agent received a 0.0 reward for thousands of steps at the beginning. It had to explore completely at random until it accidentally hit the goal and learned what to do.
- **Dead Ends:** Water holes stop the game. The agent had to learn to highly penalize any direction that led to an H tile so it would never repeat that bad move.

Conclusion

A complete Reinforcement Learning solution from first principles was successfully developed and deployed without relying on any external packages like Gymnasium or Stable Baselines. By building the custom FrozenLakeEnv grid environment and the core Q-learning updates entirely by hand, we proved that tabular temporal difference algorithms can solve complex layouts when configured correctly. Both of the trained agents successfully solved the 8x8 Frozen Lake grid world, achieving a flawless 100% success rate during final evaluation testing. However, the experimental comparison showed that the Decaying Epsilon strategy was far superior to the static framework, offering much better training stability and faster path convergence by systematically reducing unnecessary mistakes. Ultimately, this project highlights how balancing exploration and exploitation allows simple tabular agents to master sparse-reward landscapes efficiently.